Digital Systems

Exercise 1

CONTENT

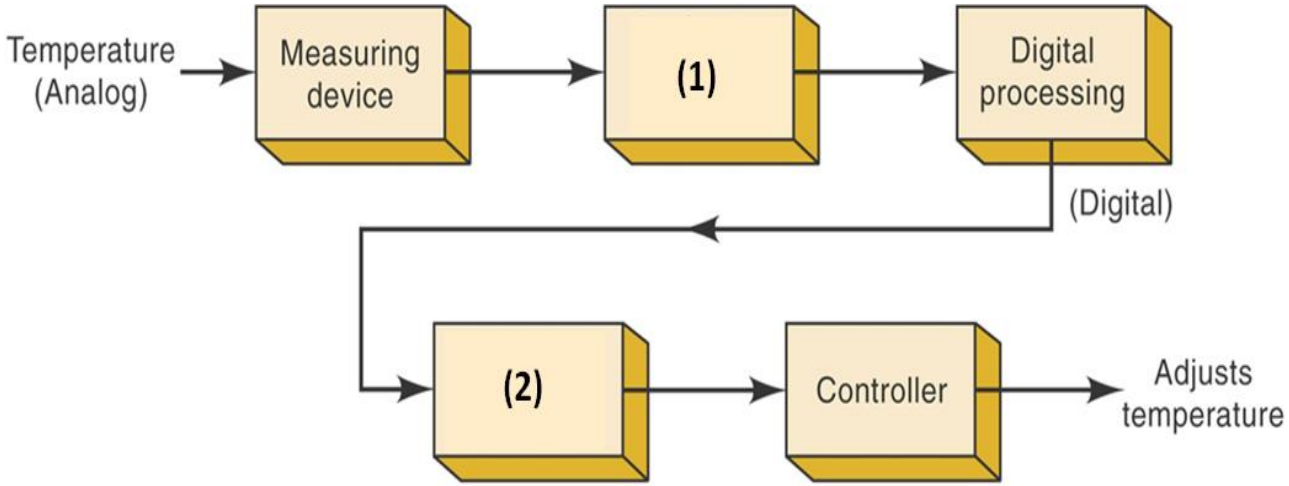
- **Number System and Codes**
  - Binary, Octal, Decimal, Hexadecimal
  - Conversions between number systems
  - Codes: Gray, Alphanumeric Codes
  - Parity Method for Error Detection
- **Logic Gates and Boolean Algebra**
  - Boolean Laws and Theorems
  - Basic gates: AND, OR, NOT, NOR, NAND,…
  - Simplifying Boolean expressions using Boolean Algebra
- **Combinational Logic Circuits**
  - Sum-of-Product (SOP), Product-of-Sum (POS)
  - Simplifying Logic Circuits using K-map method
  - Designing Combinational Logic Circuits

Chapter 1

NUMBER SYSTEM AND CODES

Number System and Codes

1.1 Choose the right answer to complete the diagram for a precision temperature regulation system.



A. (2) Digital-to-analog converter – (1) Analog-to-digital converter
B. (2) Analog-to-analog converter – (1) Digital-to-digital converter
C. (2) Analog-to-digital converter – (1) Digital-to-analog converter
D. (2) Digital-to-digital converter– (1) Analog-to-analog converter

Number System and Codes

1.2. Which of the following is the most widely used alphanumeric code for computer input and output?

A. Gray

B. Parity

C. BCD

D. ASCII

F F 2

1.3. Convert binary 111111110010_2 to hexadecimal.

A. $EE2_{16}$

B. $FF2_{16}$

C. $2FE_{16}$

D. $FD2_{16}$

1.4. How many binary digits are required to count to 100_{10}?

A. 7

B. 2

C. 3

D. 100

$$\lceil \log_2 100 \rceil = 7$$

Number System and Codes

1.5. A binary number's value changes most drastically when the _____ is changed.

A. MSB

B. Frequency

C. LSB

D. Duty Cycle

1.6. Digital electronics is based on the _____ numbering system?

A. Decimal

B. Octal

C. Binary

D. Hexadecimal

1.7. The octal value corresponding to $CF.1A_{16}$?

A. 617.032

B. 613.034

C. 633.062

D. 317.064

Number System and Codes

1.8. Find the address range of a memory. Known that the memory size is 16Kb, the size of each memory cell is 1 Byte, and the starting of memory address is 0:

Total Memory Cell = (16 x 1024) / 1 = 16384_{10} = 4000_{16}

A. 0000-FFFF

B. 0000-1FFF

C. 0000- 7FFF

D. 0000-3FFF

1.9. If a typical PC uses a 20-bit address code, the size of each memory cell is 1 Byte. How much memory can the CPU address?

A. 1MB

B. 10MB

C. 20MB

D. 580MB

Memory Size= (2^{20} x 1) byte = 1MB

Number System and Codes

1.10. Assign the proper odd parity bit to the code 111001

A. **1**111011

B. **1**111001

C. **0**111111

D. **0**011111

1.11. Assign the proper even parity bit to the code 1100001

A. **1**1100001

B. **1**100001

C. **0**1100001

D. **0**1110101

Chapter 2

LOGIC GATES AND BOOLEAN ALGEBRA

Boolean Laws & Theorems

1.	Law of Identity	$A = A$ $\overline{A} = \overline{A}$
2.	Commutative Law	$A \cdot B = B \cdot A$ $A + B = B + A$
3.	Associative Law	$A \cdot (B \cdot C) = A \cdot B \cdot C$ $A + (B + C) = A + B + C$
4.	Idempotent Law	$A \cdot A = A$ $A + A = A$
5.	Double Negative Law	$\overline{\overline{A}} = A$
6.	Complementary Law	$A \cdot \overline{A} = 0$ $A + \overline{A} = 1$
7.	Law of Intersection	$A \cdot 1 = A$ $A \cdot 0 = 0$
8.	Law of Union	$A + 1 = 1$ $A + 0 = A$
9.	DeMorgan's Theorem	$\overline{AB} = \overline{A} + \overline{B}$ $\overline{A + B} = \overline{A}\,\overline{B}$
10.	Distributive Law	$A \cdot (B + C) = (A \cdot B) + (A \cdot C)$ $A + (BC) = (A + B) \cdot (A + C)$
11.	Law of Absorption	$A \cdot (A + B) = A$ $A + (AB) = A$
12.	Law of Common Identities	$A \cdot (\overline{A} + B) = AB$ $A + (\overline{A}B) = A + B$

Logic Gates and Boolean Algebra

2.1. The output of a logic gate is 1 when all its inputs are at logic 0. The gate is either

A. a NAND or an EX-OR

~~B. an OR or an EX-NOR~~

~~C. an AND or an EX-OR~~

D. a NOR or an EX-NOR

2.2. One of the DeMorgan's theorems shows the equivalence of

A. OR gate and Exclusive OR gate

B. NOR gate and Bubbled AND gate

C. NOR gate and NAND gate

D. NAND gate and NOT gate

$\overline{A+B} = \overline{A}\,\overline{B}$

NOR

(A+B)' = A'B'

Logic Gates and Boolean Algebra

2.3. A universal logic gate is one, which can be used to generate any logic function. Which of the following is a universal logic gate?

A. OR B. AND C. XOR D. NAND

2.4. The expression for Absorption law is given by

_____

A. $A + AB = A$

B. $A + AB = B$

C. $AB + AA' = A$

D. $A + B = B + A$

Logic Gates and Boolean Algebra

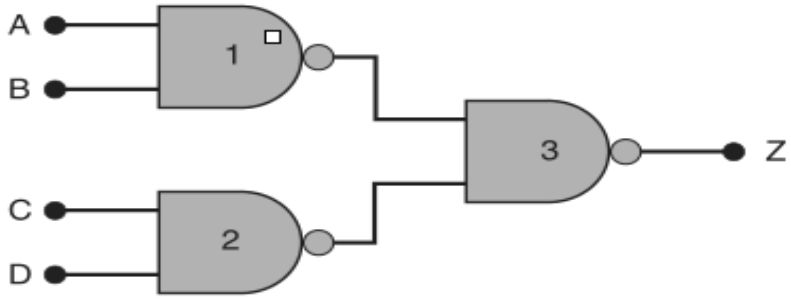
2.5. Which images is suitable the most with this description: "Output Z will go LOW only when A or B is LOW and C or D is LOW".

A. a

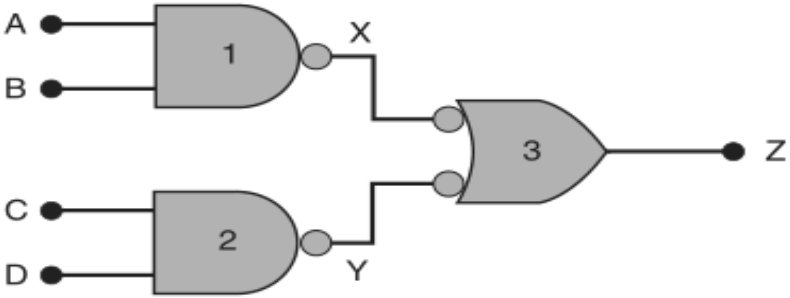
B. b

C. c

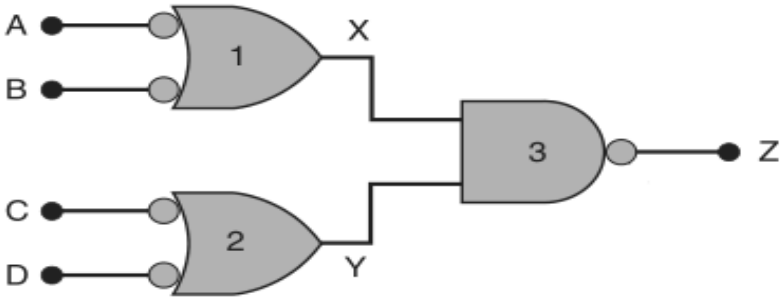
D. All of them



(a)

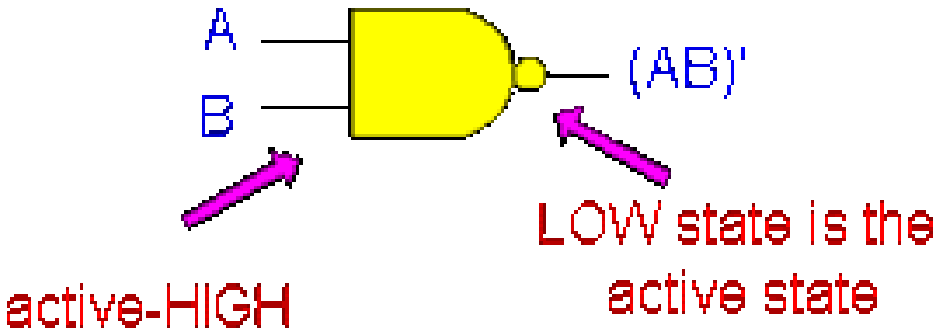


(b)

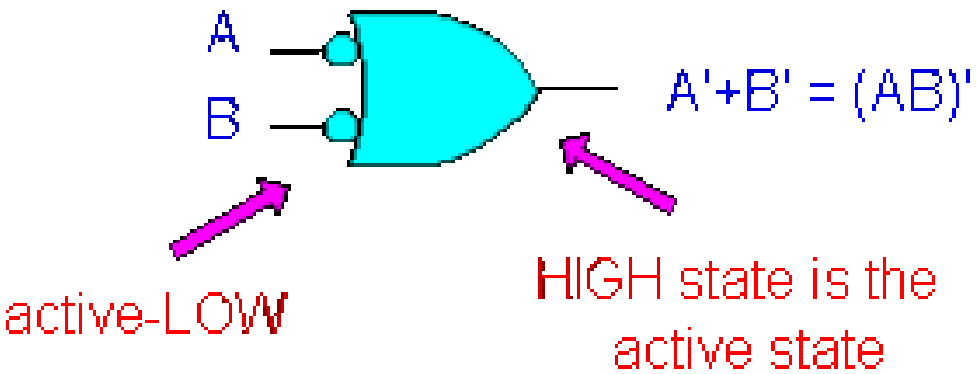


(c)

Logic Gates and Boolean Algebra



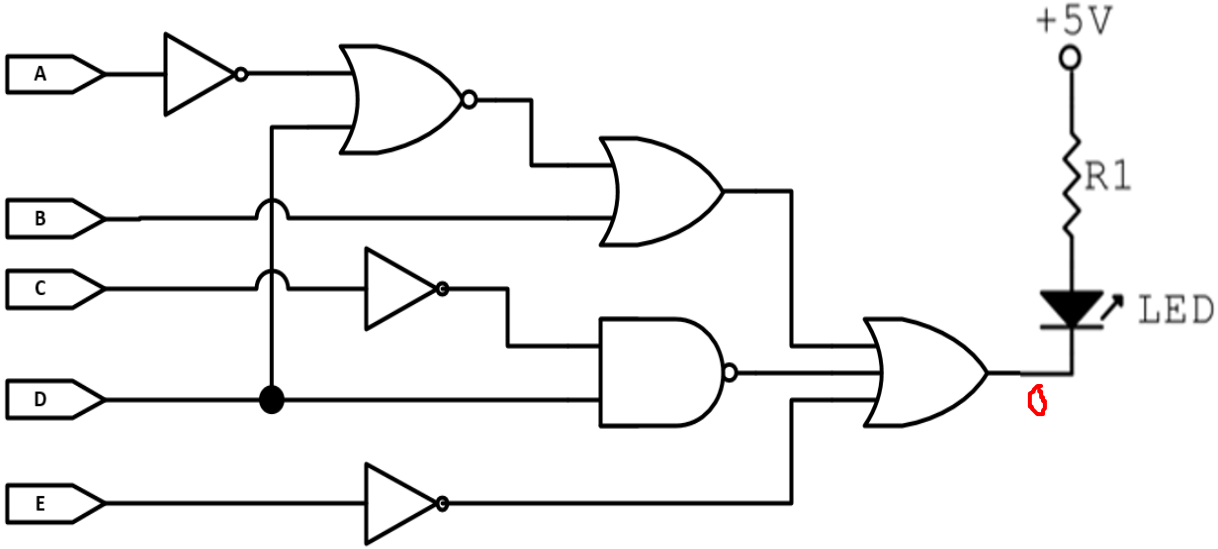
Output goes LOW only when all inputs are HIGH



Output is HIGH when any input is LOW

Logic Gates and Boolean Algebra

2.6. Define the combination of inputs for the circuit to make LED ON:



A. LED ON when B = 0 and C = 0 and D = 1 and E = 1

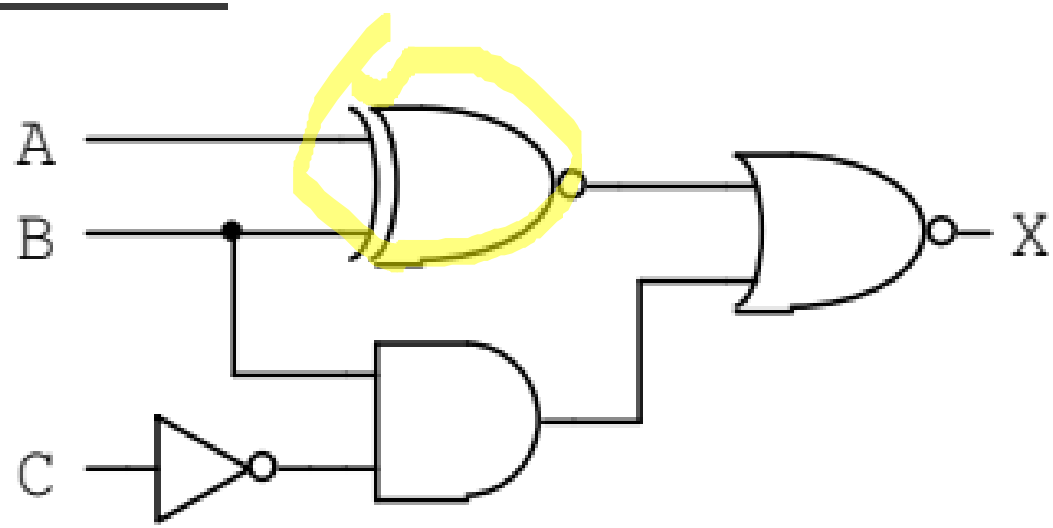
B. LED ON when A = 1 and B = 0 and C = 0 and D = 0 and E = 1

~~C. LED ON when A = 0 and B~~ = 0 and C = 0 and D = 1 and ~~E = 0~~

~~D. LED ON when A = 1 and B = 0~~ and C = 1 and D = 0 and ~~E = 0~~

Logic Gates and Boolean Algebra

2.7. Find Boolean expression of the following logic circuit_____



A. $X = \overline{A\bar{B} + \bar{A}B + B\bar{C}}$

B. $X = \overline{(\overline{A \oplus B}) + B\bar{C}}$

C. $X = (\bar{A}\bar{B} + AB)(\bar{B} + \mathrm{C})$

D. All of them

Combinational Logic Circuits

$$F = B.\overline{C} + D(\overline{\overline{B} + \overline{C}} + A.\overline{B}) + \overline{A + B + C}$$

2.8. Which of the following Boolean expressions NOT equal to F

A. $B.\overline{C} + B.C.D + A.\overline{C} + A.\overline{B}.D$

B. $B.\overline{C} + B.D + \overline{A}.\overline{B}.\overline{C} + A.\overline{B}.D$

C. $B.\overline{C} + B.D + \overline{A}.\overline{C} + A.D$

D. $B.\overline{C} + B.C.D + \overline{A}.\overline{B}.\overline{C} + A.\overline{B}.D$

Combinational Logic Circuits

$$F = B.\overline{C} + D(\overline{\overline{B} + \overline{C}} + A.\overline{B}) + \overline{A + B + C}$$

= BC′ + D(BC + AB′) + A′B′C′

= BC′ + DBC + DAB′ + A′B′C′ => D

= B(C′ + DC) + DAB′ + A′B′C′

= B(C′ + D) + DAB′ + A′B′C′ -> Common Identities

= BC′ + BD + DAB′ + A′B′C′ => B

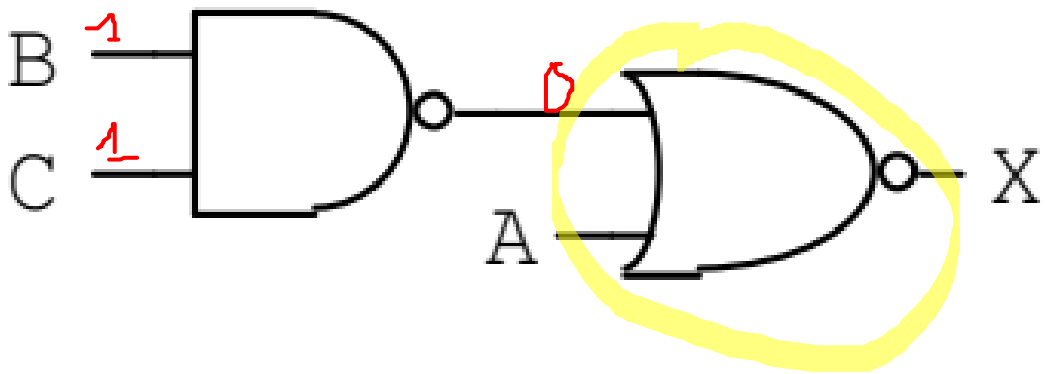
= C′(B + A′B′) + D(B + AB′)

= C′(B + A′) + D(B + A) -> Common Identities

= C′B + C′A′ + BD + AD => C

Logic Gates and Boolean Algebra

2.9. Choose the TRUE statement:



A. X = A′ when B = 1 or C = 1, otherwise X = 1
~~B. X = A when B = 0 and C = 0, otherwise X = 0~~
C. X = A′ when B = 1 and C = 1, otherwise X = 1
D. All is not correct

Logic Gates and Boolean Algebra

2.10. Given F = A'B + (B+D)'B'D , the reverse expression of F is:

A. F' = AB'

B. F' = A + B'

C. F' = AB

D. Tất cả đều sai

F = A'B + (B+D)'B'D

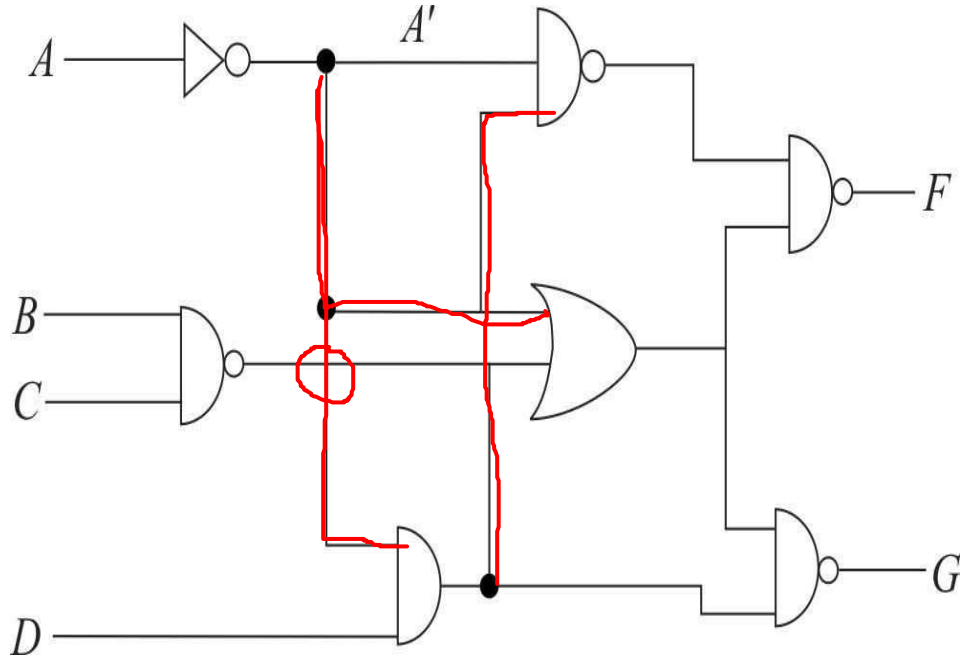
= A'B + B'D'B'D (DeMorgan)

= A'B + (B'B')(D'D) = **A'B**

=> F' = (A'B)' = A + B'

Logic Gates and Boolean Algebra

2.11. Simplifying the circuit below



- $F(A, B, C, D) = ((A'.A'D)'\ (A' + (BC)'))' = A'\ .A'D + (A' + (BC)')'$
  $= A'D + (A' + (BC)')'$ **$= A'D + ABC$**
- $G(A, B, C, D) = ((A'D)\ (A' + (BC)'))'$
  $= (A'D)' + (A' + (BC)')'$
  $= (A + D') + ABC$
  $= A\ (1 + BC) + D'$ **$= A + D'$**

Logic Gates and Boolean Algebra

2.12. Use the rules of Boolean Algebra, simplify the following:

$$F(X,Y,Z) = X(Y+Z) + Z\overline{((\overline{YX})Y)}$$

=X(Y+Z)+Z(YX+Y′)=XY+~~XZ~~+ZX+ZY′

= XY+XZ +ZY′

= XY + XZ(Y+Y′) + ZY′

= XY + XYZ + XZY′ + ZY′

= XY(1+Z) + ZY′(1+X) = 1

=**XY+ZY′** (consensus)

Logic Gates and Boolean Algebra

2.13 Simplifying the Boolean expressions

$$g = (ab' + b(c' + (c\,d)'))'$$

$$g = (ab')'\,(b\,(c' + (c\,d)')'$$

$$= (a'+b)\,(b'+ (c''\,(c\,d)'')$$

$$= (a'+b)\,(b'+ cd) = b'a'+ a'cd+cdb \text{ (consensus)}$$

$$= a'\,b' + bcd$$

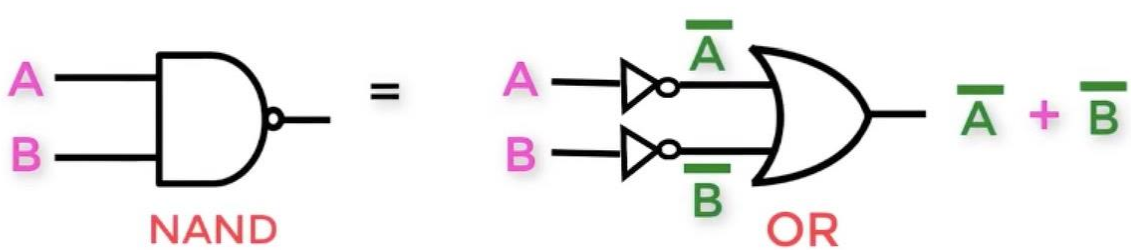
Logic Gates and Boolean Algebra

2.14. The following function is in minimum sum of products form. Implement it using only two-input NAND gates.

- $K = \bar{A}\,\bar{B}\,X + A\bar{B}\,\bar{X} + \bar{A}B\bar{X} + ABX$

$$K = \bar{A}\,\bar{B}\,X + A\bar{B}\,\bar{X} + \bar{A}B\bar{X} + ABX$$

$$= \underbrace{\underbrace{(\underbrace{\bar{A}\,\bar{B}}_{\text{AND}} + \underbrace{AB}_{\text{AND}})}_{\text{OR}}X}_{\text{AND}} + \underbrace{\bar{X}\,\underbrace{(\underbrace{A\bar{B}}_{\text{AND}} + \underbrace{\bar{A}B}_{\text{AND}})}_{\text{OR}}}_{\text{AND}}$$



Logic Gates and Boolean Algebra

2.15. The following function is in minimum sum of products form. Implement it using only two-input NAND gates.

- $F = W\overline{Y} + WX\overline{Z} + \overline{Y}Z + \overline{W}\overline{X}Z$

  $=$

Chapter 3

COMBINATIONAL LOGIC CIRCUITS

Combinational Logic Circuits

1. Sum of Products – SoP:

- A SoP expression will appear as two or more AND terms Ored together
- E.g.: $ABC + A'B'C$

  $AB + A'BC' + C'D' + D$

2. Product of Sums – PoS:

- A PoS expression will appear as two or more OR terms ANDed together
- E.g.: $(A + B' + C)(A + C)$

  $(A + B')(C' + D)$

Canonical forms

❑ Canonical forms

- Standard forms for a Boolean expression
- Provide unique algebraic signatures for an expression
- Generally are not the simplest form (can be minimized)
- Derive directly from a Boolen function's truth table

❑ Two canonical forms:

- Sum-of-Products (SoP – minterms)
- Product-of-Sums (PoS – maxterms)

Sum-of-Products canonical form

- **Also known as the disjunctive normal form**
  - Commonly called a minterm expansion

minterm

001 011 101 110 111

F = A'B'C + A'BC + AB'C + ABC' + ABC

A	B	C	F	F'
0	0	0	0	1
0	0	1	1	0
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	0

F' = A'B'C' + A'BC' + AB'C'

Minterms

- Variables appears exactly once in each minterm, in true or inverted form (but not both)

A	B	C	minterms
0	0	0	A'B'C' m0
0	0	1	A'B'C m1
0	1	0	A'BC' m2
0	1	1	A'BC m3
1	0	0	AB'C' m4
1	0	1	AB'C m5
1	1	0	ABC' m6
1	1	1	ABC m7

short-hand notation for minterms of 3 variables

F in canonical form:

$$F(A,B,C) = \Sigma m(1,3,5,6,7)$$
$$= m1 + m3 + m5 + m6 + m7$$
$$= A'B'C + A'BC + AB'C + ABC' + ABC$$

canonical form → minimal form

$$F(A,B,C) = A'B'C + A'BC + AB'C + ABC + ABC'$$
$$= (A'B' + A'B + AB' + AB)C + ABC'$$
$$= ((A' + A)(B' + B))C + ABC'$$
$$= C + ABC'$$
$$= ABC' + C$$
$$= AB + C$$

Product-of-sums canonical form

- **Also known as the conjunctive normal form**
  - Commonly called a maxterm expansion

maxterm

000 010 100

$$F = (A + B + C)\ (A + B' + C)\ (A' + B + C)$$

A	B	C	F	F'
0	0	0	0	1
0	0	1	1	0
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	0

$$F' = (A+B+C')(A+B'+C')(A'+B+C')(A'+B'+C)(A'+B'+C')$$

Maxterms

- Variables appears exactly once in each maxterm, in true or inverted form (but not both)

A	B	C	maxterms	
0	0	0	A+B+C	M0
0	0	1	A+B+C'	M1
0	1	0	A+B'+C	M2
0	1	1	A+B'+C'	M3
1	0	0	A'+B+C	M4
1	0	1	A'+B+C'	M5
1	1	0	A'+B'+C	M6
1	1	1	A'+B'+C'	M7

short-hand notation for maxterms of 3 variables

F in canonical form:

$$F(A,B,C) = \Pi M(0,2,4)$$
$$= M0 \bullet M2 \bullet M4$$
$$= (A+B+C)(A+B'+C)(A'+B+C)$$

canonical form → minimal form

$$F(A,B,C) = (A+B+C)(A+B'+C)(A'+B+C)$$
$$= (A+B+C)(A+B'+C)\bullet (A+B+C)(A'+B+C)$$
$$= (A + C)\,(B + C)$$

Conversion between canonical forms

- **Minterm to Maxterm**
  - Use maxterms that do not appear in minterm expansion
  - E.g., F(A,B,C) = $\sum m(1,3,5,6,7) = \prod M(0,2,4)$
- **Maxterm to Minterm**
  - Use minterms that do not appear in maxterm expansion
  - E.g., F(A,B,C) = $\prod M(0,2,4)$ = $\sum m(1,3,5,6,7)$
- **Minterm expansion of F to minterm expansion of F'**
  - Use minterms that do not appear
  - E.g., F(A,B,C) = $\sum m(1,3,5,6,7)$ => **F'(A,B,C) = $\sum \boldsymbol{m(0,2,4)}$**
- **Maxterm expansion of F to maxterm expansion of F'**
  - Use maxterms that do not appear
  - E.g., F(A,B,C) =$\prod M(0,2,4)$ => **F'(A,B,C) =**$\prod M(1,3,5,6,7)$

Combinational Logic Circuits

x	y	z	Minterms	Maxterms
0	0	0	$m_0 = \bar{x}\,\bar{y}\,\bar{z}$	$M_0 = x + y + z$
0	0	1	$m_1 = \bar{x}\,\bar{y}\,z$	$M_1 = x + y + \bar{z}$
0	1	0	$m_2 = \bar{x}\,y\,\bar{z}$	$M_2 = x + \bar{y} + z$
0	1	1	$m_3 = \bar{x}\,y\,z$	$M_3 = x + \bar{y} + \bar{z}$
1	0	0	$m_4 = x\,\bar{y}\,\bar{z}$	$M_4 = \bar{x} + y + z$
1	0	1	$m_5 = x\,\bar{y}\,z$	$M_5 = \bar{x} + y + \bar{z}$
1	1	0	$m_6 = x\,y\,\bar{z}$	$M_6 = \bar{x} + \bar{y} + z$
1	1	1	$m_7 = x\,y\,z$	$M_7 = \bar{x} + \bar{y} + \bar{z}$

Combinational Logic Circuits

2. Find the minterm expansion of the following expression:

$$f = a'(b'+d)+acd'$$

$f(a,b,c,d) = a'b' + a'd + acd'$

$= a'b'(c+c')(d+d') + a'd(b+b')(c+c') + acd'(b+b')$

$= \mathbf{a'b'c'd' + a'b'c'd + a'b'cd' + a'b'cd + a'b'c'd +}$

$\mathbf{a'b'cd + a'bc'd + a'bcd + abcd' + ab'cd'}$

Karnaugh Map Technique

K-Maps, like truth tables, are a way to show the relationship between logic inputs and desired outputs.

K-Maps are a graphical technique used to simplify a logic equation.

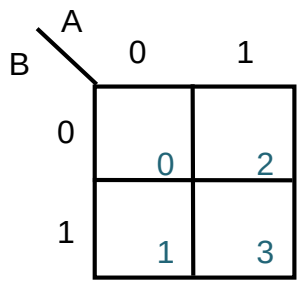
K-Maps are very procedural and much *cleaner* than Boolean simplification.

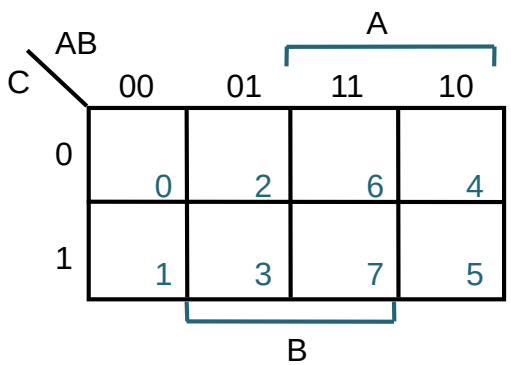
K-Maps can be used for any number of input variables, BUT are only practical for fewer than six.

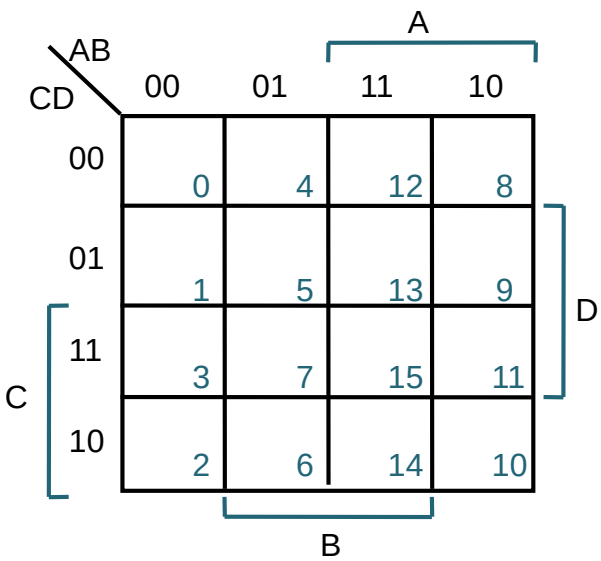
K-Map Format

Each minterm (or maxterm) in a truth table corresponds to a cell in the K-Map.

K-Map cells are labeled so that both horizontal and vertical movement **differ only in one variable.**







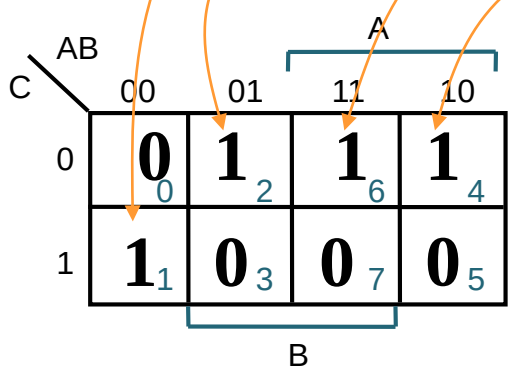
Numbering Scheme: 00, 01, 11, 10

Truth Table -TO- K-Map

$f_1(A,B,C) = m_1 + m_2 + m_4 + m_6$

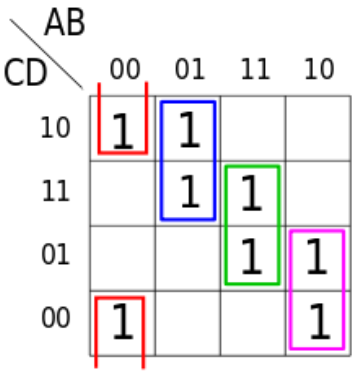
$= A'B'C + A'BC' + AB'C' + ABC'$

A	B	C		f_1
0	0	0	m0	0
0	0	1	m1	1
0	1	0	m2	1
0	1	1	m3	0
1	0	0	m4	1
1	0	1	m5	0
1	1	0	m6	1
1	1	1	m7	0

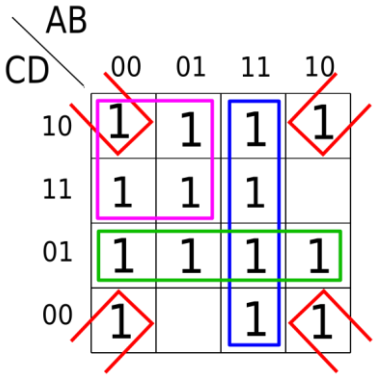


Groupings

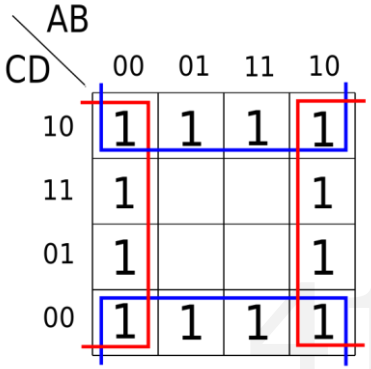
Grouping a pair of adjacent 1's eliminates the variable that appears in complemented and uncomplemented form.



Grouping a quad of 1's eliminates the two variables that appear in both complemented and uncomplemented form.



Grouping an octet of 1's eliminates the three variables that appear in both complemented and uncomplemented form, etc.....



Kmap

Use Karnaugh maps to reduce function to minimum SOP forms:

F(ABCD) = ∑ m (1, 5, 6, 7, 9, 11, 15) + ∑ d (0, 2, 3, 8, 12, 14); **D = LSB.**

a) Draw K-maps to reduce F

b) Write all the minimum sum-of-product forms (SOPs) of the function F

c) Implement a minimized circuit F using only two-input NAND gates